

CV Matcher

Studienprojektgruppe unter der Leitung von FH-Assistenzprofessorin Susanne Schaller, MMSc (susanne.schaller@fh-hagenberg.at), Mitglieder:

- Lukas Steiniger, Projektleitung
- Nina Schellner, Projektleitung-Stv.
- Fatemeh Aghaei
- Sigrid Klein
- Maja Nikolic

Inhalt

1	Einführung und Problemstellung.....	1
2	Installation und Einrichtung	1
2.1	Voraussetzungen	1
2.2	Source Code	1
2.3	Installation.....	2
2.4	Initialisierung der Datenbank.....	2
2.5	Applikation starten	2
2.6	Applikation beenden.....	2
3	Dokumentation der Anwendung	2
4	Technische Dokumentation.....	4
4.1	Parsing	4
4.1.1	Large Language Model	4
4.1.2	ESCO Translate/Labelling.....	6
4.1.3	ESCO Enrich.....	6
4.2	Matching.....	6
4.2.1	ESCO.....	7
4.2.2	BERT/JobBERT	7
4.2.3	Matching mittels LLM.....	8

1 Einführung und Problemstellung

Das Studienprojekt wurde in Zusammenarbeit mit der Firma TRESCON durchgeführt. Die Dauer des gesamten Studienprojektes erstreckte sich über drei Semester (01.10.2024-14.01.2026).

Der manuelle Vergleich von Stellenausschreibungen und Lebensläufen ist ressourcenintensiv und fehleranfällig. Ziel des Studienprojektes war es daher, eine zuverlässige und datenschutzkonforme Anwendung zu entwickeln, die für AnwenderInnen eine Vorauswahl der KandidatInnen liefert. Die Vorauswahl basiert hauptsächlich auf den vorherigen Berufserfahrungen, der Ausbildung und den technischen und sozialen Fähigkeiten der BewerberInnen. Das Ergebnis soll ein Score sein, der die KandidatInnen reiht und sie untereinander vergleichbar macht.

2 Installation und Einrichtung

2.1 Voraussetzungen

Am Gerät muss Docker Desktop¹ installiert sein.

2.2 Source Code

Der Source Code muss vom Repository² heruntergeladen werden. Dafür muss am grünen „Code“-Button „Download ZIP“ ausgewählt werden:

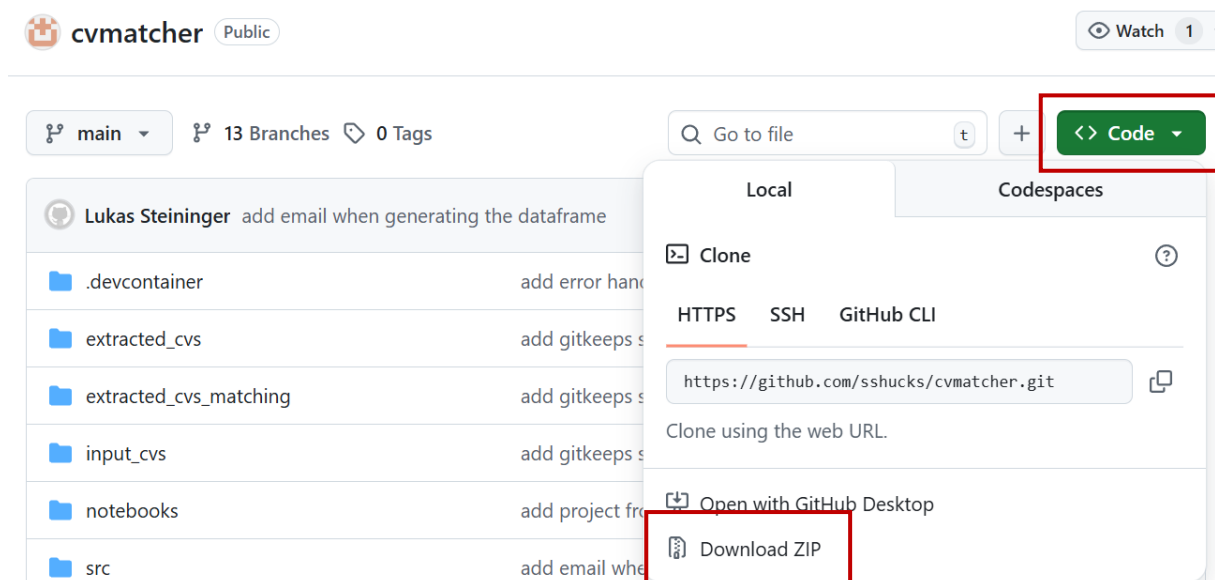


Abbildung 1: Download ZIP im Repository

Das ZIP muss anschließend noch in den gewünschten Ordner entpackt werden.

¹ <https://docs.docker.com/get-started/introduction/get-docker-desktop/>

² <https://github.com/sshuks/cvmatcher>

2.3 Installation

Für alle nachfolgenden Schritte muss Docker Desktop im Hintergrund aktiv laufen.

Für die Installation muss 1_installer.bat gestartet werden. Diese und alle nachfolgenden Anwendungen sind im Ordner „installer-scripts“ zu finden. Dieser Schritt muss nur bei der erstmaligen Installation durchgeführt werden. Bis alle Abhängigkeiten installiert wurden, kann es bis zu 20 Minuten dauern. Sobald dieser Schritt abgeschlossen ist, können die nächsten Schritte durchgeführt werden.

2.4 Initialisierung der Datenbank

Die Anwendung speichert geparte Lebensläufe ab, um zeit- und ressourcenaufwändig Berechnung so minimal wie möglich zu halten. Dazu wird unter anderem eine SQLite Datenbank verwendet. Vor der Inbetriebnahme der Software muss diese initialisiert werden. Dazu kann die Datei 1_2_create_database.bat ausgeführt werden. Alternative, kann im Docker-Container der Befehl „python caching/database.py“ verwendet werden.

Die Installation und die Initialisierung der Datenbank muss nur einmalig für die Einrichtung ausgeführt werden.

2.5 Applikation starten

Mit 2_start_application.bat wird die Anwendung und der Container gestartet.

2.6 Applikation beenden

Um die Applikation zu beenden, muss das Browserfenster geschlossen werden und anschließend 3_stop_application.bat gestartet werden. Das beendet auch den Container.

3 Dokumentation der Anwendung

Die Anwendung besteht aus insgesamt drei Seiten:

- Home
- Analysis
- Settings

Die Hauptseite (s. Abbildung 2) enthält die wichtigsten Informationen im Überblick. In der linken Spalte können die Dokumente hochgeladen werden. Dabei kann mithilfe eines Schalters eingestellt werden, ob die im Hintergrund liegende Datenbank vollständig verwendet werden soll, oder nicht. Weiters können auch die Gewichtungen für die vier Bereiche eingestellt werden.

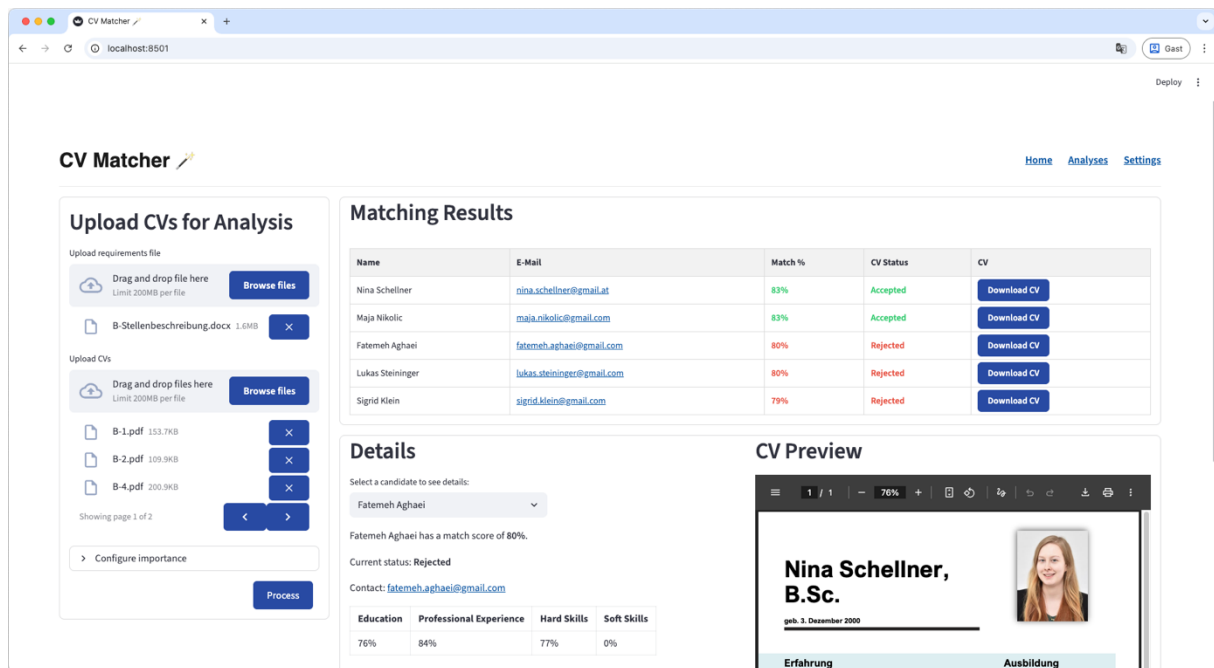


Abbildung 2: Home-Seite der Anwendung

In der rechten Spalte werden die Ergebnisse des Matchings dargestellt. Im oberen Teil werden die KandidatInnen absteigend nach ihrer Übereinstimmung mit der Stellenausschreibung gereiht. Zusätzlich findet sich eine Schaltfläche, um die Person direkt per Mail kontaktieren zu können oder den Lebenslauf zu downloaden. Im unteren Bereich findet sich eine Detailansicht, wo genauere Informationen zu den einzelnen Scores in den Kategorien, wie auch eine Preview des Lebenslaufes abgebildet sind.

Die Analyseseite (s. Abbildung 3) zeigt eine Historie der vergangenen Analysen. Die Tabelle links zeigt, wann für welche Ausschreibung wie viele Lebensläufe analysiert wurden.

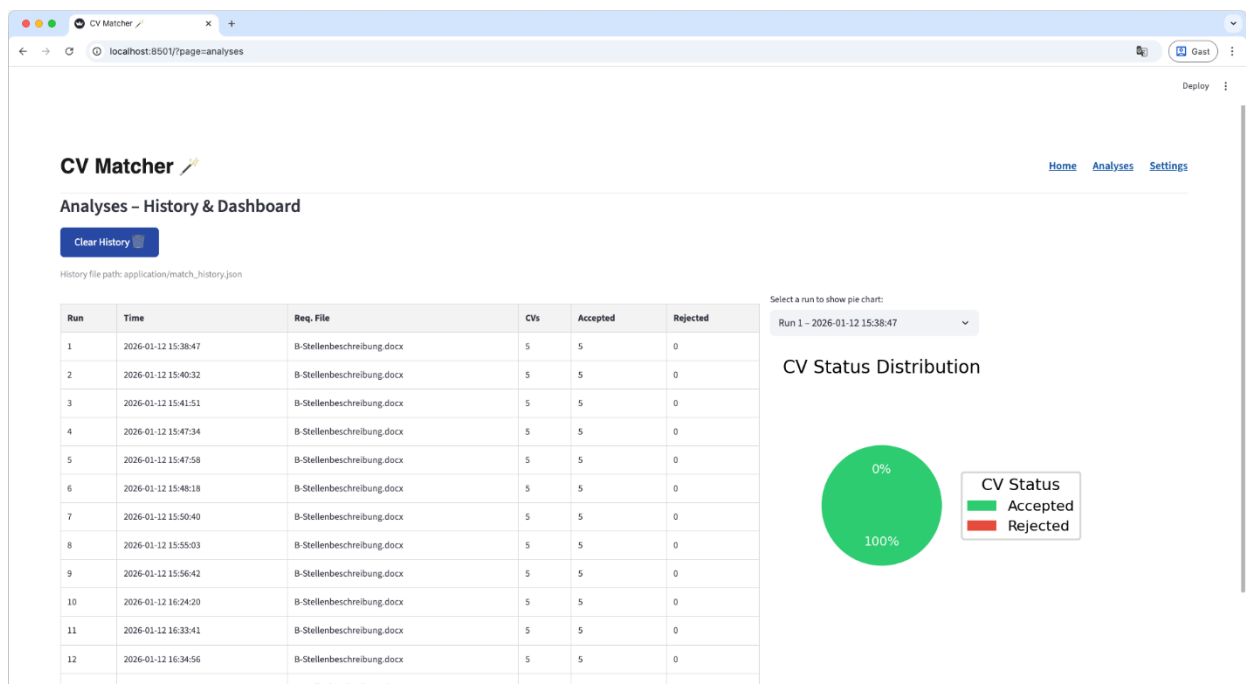


Abbildung 3: Analyse-Seite der Anwendung

Weiters wird dargestellt, wie viele davon als „Accepted“ und wie viele als „Rejected“ eingestuft wurden. Das Piechart rechts zeigt den im Dropdown-Menü ausgewählten Durchlauf an.

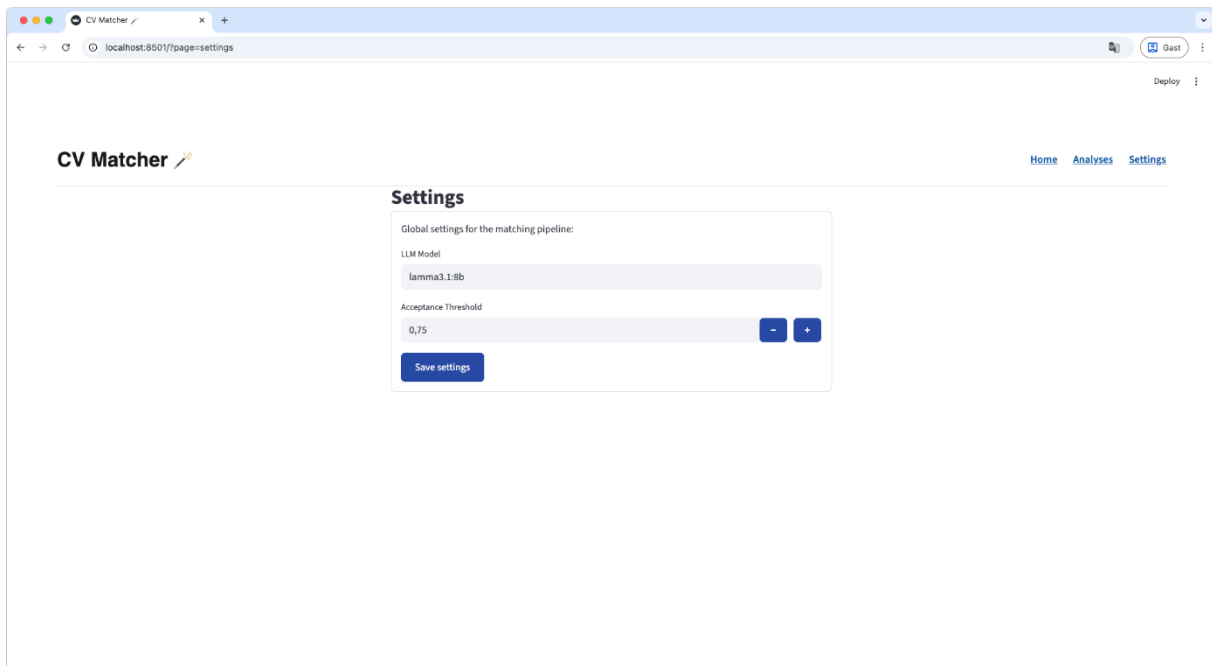


Abbildung 4: Settings-Seite der Anwendung

Auf der Settings-Seite (s. Abbildung 4) der Anwendung können Parameter eingestellt werden, die global für die Analyse verwendet werden. Aktuell befinden sich hier nur zwei einstellbare Parameter. Der erste beschreibt, welches Large Language Model für das Parsing (und Matching) verwendet werden soll, während der zweite die Acceptance Threshold der Hauptseite einstellt. Damit kann geregelt werden, ab welchem Score die KandidatInnen als „Accepted“ eingestuft werden.

4 Technische Dokumentation

Die Applikation kann mit Lebensläufen und Anforderungsdokumenten im .docx und .pdf Format verwendet werden. Dabei können sowohl englische als auch deutsche Dokumente genutzt werden. Diese werden anschließend geparkt und miteinander verglichen. Im Folgenden werden die einzelnen Methoden näher beschrieben.

4.1 Parsing

4.1.1 Large Language Model

Für das Parsen der Lebensläufe und der Stellenausschreibungen wird ein Large Language Model (LLM) verwendet. Dabei wird eine zuvor definierte Anweisung an das LLM übergeben. Dieses definiert für die Lebensläufe, dass das LLM ein hilfreicher Assistent ist, der auf das Extrahieren von strukturierten Informationen aus Lebensläufen spezialisiert ist. Weiters wird ein fixes Schema vorgegeben, welchem der Output folgen muss, und die

Sprache. Im Fall der Stellenausschreibungen wird das LLM als Experte für die Analyse von Stellenausschreibungen genutzt. Es wird ebenfalls die Sprache, wie auch das fixe Output-Schema mitgegeben.

Sowohl für das Parsing der Lebensläufe als auch der Stellenbeschreibung wird anschließend ein Majority-Voting durchgeführt. Dabei kann definiert werden, wie viele Antworten für ein Dokument insgesamt generiert werden sollen und wie viele Antworten den gleichen Inhalt enthalten müssen, damit dieser Inhalt in den finalen Output übernommen wird.

Das Endergebnis der Lebensläufe ist ein .json File, das strikt die folgenden Informationen enthält:

- Persönliche Informationen:
 - Name
 - E-Mail
 - Telefonnummer
 - Geburtsdatum
- Technische Kompetenzen (jobspezifisch)
- Soziale Kompetenzen
- Ausbildung:
 - Ausbildungsbereich
 - Indikator, ob die Ausbildung abgeschlossen wurde
 - Ausbildungsgrad, zB. Bachelor of Science
- Berufserfahrung:
 - Start- und Enddatum
 - Branche bzw. Bereich des Unternehmens
 - Jobtitel
 - Verantwortungsbereiche

Die Stellenausschreibung wird nach dem Parsen ebenfalls als .json gespeichert und enthält folgende Informationen:

- Jobtitel der Ausschreibung
- Benötigte bzw. gewünschte Berufserfahrung:
 - Jobtitel
 - Branche bzw. Bereich des Unternehmens
 - Beschäftigungsdauer
- Ausbildung:
 - Ausbildungsbereich
 - Ausbildungsgrad, zB. Bachelor of Science
- Technische Kompetenzen
- Soziale Kompetenzen

Diese Dokumente dienen als Grundlage für die weiteren Verarbeitungsschritte. Außerdem werden sie in der Datenbank zwischengespeichert.

4.1.2 ESCO Translate/Labelling

ESCO ist eine Ontologie der Europäischen Union, die europaweit eine einheitliche Struktur für Berufe, Kompetenzen und Qualifikationen garantiert. Es ist festgelegt, welche Fähigkeiten zu welchen Berufen gehören und wie diese miteinander zusammenhängen. Dabei werden die Berufe in einer hierarchischen Struktur dargestellt, die einen mehrstufigen Vergleich ermöglicht.

Um nicht standardisierten Berufs-/Fähigkeitsbezeichnungen die entsprechenden ESCO-Begriffe zuordnen zu können, müssen diese mit den ESCO-Begriffen verglichen werden. Dazu werden alle ESCO-Begriffe mitsamt deren definierten Synonymen in einen hochdimensionalen semantischen Vektorraum überführt (siehe Embeddings). Das gleiche wird auch mit den Begriffen aus den Lebensläufen und dem Anforderungsdokument gemacht. Über den semantischen Vektorraum kann dann die Ähnlichkeit zwischen den Begriffen und den ESCO-Begriffen berechnet werden. Ab einem vorher definierten Threshold (`esco_labelling_threshold`) wird ein Begriff als Match interpretiert. Ab diesem Zeitpunkt kann mit dem standardisierten ESCO-Begriff gearbeitet werden.

Dieser Schritt ist für das ESCO-Matching unbedingt notwendig und kann nicht ausgelassen werden.

4.1.3 ESCO Enrich

Beim ESCO Enrich geht es darum, die Zusammenhänge von ESCO zu verwenden, um die Daten im Lebenslauf zu erweitern. Dazu gehört die Ergänzung von technischen Fähigkeiten durch die Berufserfahrung und die Erweiterung von technischen und sozialen Fähigkeiten um die allgemeinen Bezeichnungen von genannten spezifischen Fähigkeiten. Außerdem können auch die Begriffe an sich mit den allgemeinen Bezeichnungen erweitert werden, was aber erst im Matching passiert.

Dieser Schritt ist grundsätzlich optional und kann als semantische Verbesserung für andere Methoden verwendet werden.

4.2 Matching

Das Matching kann folgendermaßen durchgeführt werden:

- ESCO
- BERT/JobBERT
- Matching mittels LLM

Als Ergebnis wird je Kategorie ein Score zwischen 0 und 1 zurückgegeben, der im Anschluss gewichtet gemittelt wird. Je größer dieser Gesamtscore ist, umso besser passt ein KandidatIn auf den ausgeschriebenen Job.

4.2.1 ESCO

Beim Matching mit ESCO wird geprüft, wie viele der ESCO-Begriffe im Anforderungsdokument in einem Lebenslauf abgedeckt sind. Der Score orientiert sich somit an einer einfachen Schnittmenge:

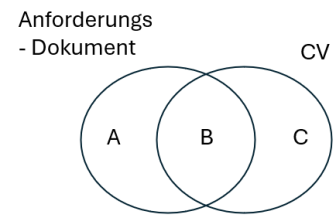


Abbildung 5: Einfaches Schema der Berechnung des ESCO-Scores

Die Abdeckung von Begriffen wird jedoch nicht binär (vorhanden oder nicht vorhanden) bewertet, sondern auch die ESCO-Hierarchie wird herangezogen. Wie viel ein Begriff eine Ebene höher wert ist, wird durch den Parameter „decay“ definiert. Ein Begriff, der exakt vorkommt, erhält den Wert 1 ($decay^0$), ein Begriff, der in der nächsthöheren Hierarchieebene vorkommt, bekommt den Wert $decay^1$, in der darauffolgenden Ebene dann $decay^2$. Das Scoring folgt damit einem exponentiellen Schema.

4.2.2 BERT/JobBERT

BERT steht für *Bidirectional Encoder Representations from Transformers*. Es basiert auf Embeddings, wo Wörter in ein numerisches Format überführt werden, mit dem Computer rechnen können. Dabei werden die Wörter einzeln als Vektoren im hochdimensionalen Raum dargestellt. Ähnliche Begriffe finden sich in ähnlichen Regionen wieder. BERT liest Texte bidirektional, das heißt vorangegangene und nachfolgende Worte werden ebenfalls berücksichtigt. Somit kann der Kontext besser verstanden werden und Wörter mit mehreren Bedeutungen korrekt und eindeutig zugeordnet werden. Sprachlich können sowohl englische als auch deutsche Texte verglichen werden, aber es muss sich auf eine Sprache eingeschränkt werden.

JobBERT ist speziell trainiert auf Jobtitel und die zugehörigen Fähigkeiten und Voraussetzungen. Somit eignet er sich gut für diesen Anwendungsfall, speziell für den Vergleich der Berufserfahrung und den technischen und sozialen Fähigkeiten. Für die Ausbildung wird auf den normalen BERT zurückgegriffen.

Für die Berechnung der Scores wird für die drei Kategorien, wo der JobBERT genutzt wird, gleich vorgegangen. Zunächst werden die Embeddings für alle Begriffe der Lebensläufe und des Anforderungsprofils erstellt. Dabei wird ein Begriff als Eintrag in einem Feld definiert, zB. ein Jobtitel. Das Embedding wird in Batches durchgeführt, was eine effizientere Bearbeitung ermöglicht. Anschließend wird die Cosine-Similarity zwischen den Anforderungen und einem Lebenslauf berechnet. Diese misst die Ähnlichkeit von zwei Vektoren (also zwei Begriffen) anhand des Winkels zwischen den Vektoren. Dabei werden alle Begriffe aus einem Lebenslauf mit allen Begriffen der Anforderungen verglichen und für jede Anforderung wird die größte Ähnlichkeit herangezogen. Um den abschließenden Score zu erhalten, wird der Mittelwert der Scores aller Anforderungen berechnet.

Technisch funktioniert der Standard-BERT für die Ausbildung genauso wie der JobBERT. Jedoch wird bei der Berechnung der Ähnlichkeit zusätzlich berücksichtigt, ob eine Ausbildung abgeschlossen wurde oder nicht. Wenn sie nicht abgeschlossen wurde, wurde ein Strafterm drauf multipliziert. Damit soll gewährleistet werden, dass eine nicht abgeschlossene Ausbildung nicht so viel zählt, wie eine abgeschlossene, aber trotzdem nicht vollständig ausgeschlossen wird.

4.2.3 Matching mittels LLM

Für das Matching wird dem LLM entweder ein Teil (entsprechend einer Kategorie) oder der gesamte Lebenslauf und Anforderungsdokument zur Verfügung gestellt. Dabei soll das LLM anhand der gegebenen Informationen in der Stellenausschreibung feststellen, ob einE KandidatIn für ein Gespräch eingeladen werden soll. Daher ist die Codierung in diesem Fall nur binär. Wenn das Matching nur auf einem Teil der Daten beruht, dann werden die einzelnen Scores im Anschluss noch gewichtet gemittelt.